

System and Network Security

Dr. Ashok Kumar Das

IEEE Senior Member
Professor

Center for Security, Theory and Algorithmic Research
International Institute of Information Technology, Hyderabad

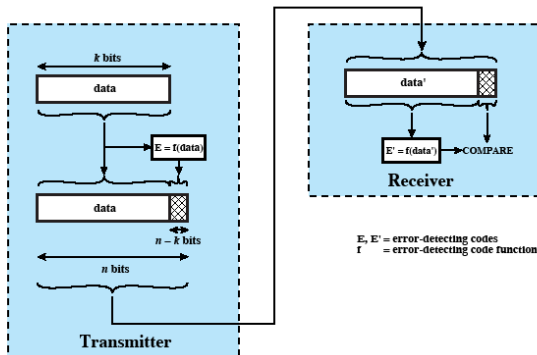
E-mail: iitkgp.akdas@gmail.com, ashok.das@iiit.ac.in

Homepage: <http://www.iiit.ac.in/people/faculty/ashok-kumar-das>

Personal Homepage: <https://sites.google.com/view/iitkgpakdas/>

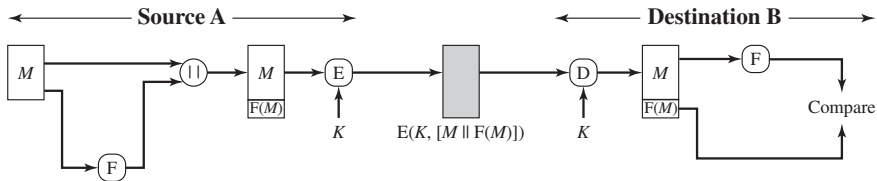
Security at the Datalink Layer

Error Detection Process

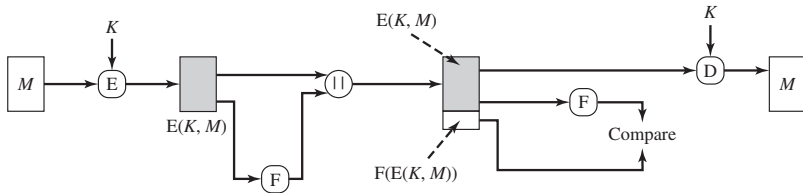


- It may be difficult to determine *automatically* if incoming ciphertext decrypts to intelligible plaintext.
- For example, if the plaintext is a binary object file or digitized X-rays, determination of properly formed and therefore authentic plaintext may be difficult.
- Thus, an opponent could achieve a certain level of disruption simply by issuing messages with random content purporting to come from a legitimate user.
- One solution to this problem is to force the plaintext to have some structure that is easily recognized but that cannot be replicated without recourse to the encryption function.
- We could, for example, append an error-detecting code, also known as a frame check sequence (FCS) or checksum, to each message before encryption.

Internal and External Error Control



(a) Internal error control



(b) External error control

M : plaintext message; F : function that produces an FCS (frame check sequence); $\parallel$: concatenation operation; $E(K, M)$: encryption of M using key K ; $D(K, M)$: decryption of M using key K ; K : shared key between source A and destination B .

- Note that the order in which the FCS and encryption functions are performed is critical.
- With internal error control, authentication is provided because an opponent would have difficulty generating ciphertext that, when decrypted, would have valid error control bits.
- If instead the FCS is the outer code, an opponent can construct messages with valid error-control codes.
- Although the opponent cannot know what the decrypted plaintext will be, he or she can still hope to create confusion and disrupt operations.

Password Management

Password Protection

- The front line of defense against intruders is the password system.
- Virtually all multiuser systems require that a user provide not only a name or identifier (ID) but also a password.
- The password serves to authenticate the ID of the individual logging on to the system. In turn, the ID provides security in the following ways:
 - ▶ The ID determines whether the user is authorized to gain access to a system. In some systems, only those who already have an ID filed on the system are allowed to gain access.
 - ▶ The ID determines the privileges accorded to the user. A few users may have supervisory or “superuser” status that enables them to read files and perform functions that are especially protected by the operating system.
 - ▶ The ID is used in what is referred to as discretionary access control. For example, by listing the IDs of the other users, a user may grant permission to them to read files owned by that user.

The Vulnerability of Passwords

- To understand the nature of the threat to password-based systems, let us consider a scheme that is widely used on UNIX, in which passwords are never stored in the clear.
- Each user selects a password of up to eight printable characters in length. This is converted into a 56-bit value (using 7-bit ASCII) that serves as the key input to an encryption routine.
- The encryption routine, known as crypt(3), is based on DES. The DES algorithm is modified using a 12-bit “salt” value. Typically, this value is related to the time at which the password is assigned to the user. The modified DES algorithm is exercised with a data input consisting of a 64-bit block of zeros. The output of the algorithm then serves as input for a second encryption. This process is repeated for a total of 25 encryptions. The resulting 64-bit output is then translated into an 11-character sequence.



The Vulnerability of Passwords

The salt serves three purposes:

- It prevents duplicate passwords from being visible in the password file. Even if two users choose the same password, those passwords will be assigned at different times. Hence, the “extended” passwords of the two users will differ.
- It effectively increases the length of the password without requiring the user to remember two additional characters.
- It prevents the use of a hardware implementation of DES, which would ease the difficulty of a brute-force guessing attack.

The Vulnerability of Passwords

There are two threats to the UNIX password scheme:

- First, a user can gain access on a machine using a guest account or by some other means and then run a password guessing program, called a password cracker, on that machine. The attacker should be able to check hundreds and perhaps thousands of possible passwords with little resource consumption.
- In addition, if an opponent is able to obtain a copy of the password file, then a cracker program can be run on another machine at leisure. This enables the opponent to run through many thousands of possible passwords in a reasonable period.

The Vulnerability of Passwords

[Alvare, A. "How Crackers Crack Passwords or What Passwords to Avoid." Proceedings, UNIX Security Workshop II, August 1990.] reports the following techniques for learning passwords:

- Try default passwords used with standard accounts that are shipped with the system. Many administrators do not bother to change these defaults.
- Exhaustively try all short passwords (those of one to three characters).
- Try words in the system's online dictionary or a list of likely passwords. Examples of the latter are readily available on hacker bulletin boards.
- Collect information about users, such as their full names, the names of their spouse and children, boyfriends and girlfriends, pictures in their office, and books in their office that are related to hobbies.
- Try users' phone numbers, Social Security numbers, and room numbers.
- Try all legitimate license plate numbers for this state.
- Use a Trojan horse to bypass restrictions on access.

Password Selection Strategies

Four basic techniques are in use:

- User education
- Computer-generated passwords
- Reactive password checking
- Proactive password checking

Password Selection Strategies

User education:

- Users can be told the importance of using hard-to-guess passwords and can be provided with guidelines for selecting strong passwords.
- This user education strategy is unlikely to succeed at most installations, particularly where there is a large user population or a lot of turnover.
- Many users will simply ignore the guidelines.

Password Selection Strategies

Computer-generated passwords:

- If the passwords are quite random in nature, users will not be able to remember them.
- Even if the password is pronounceable, the user may have difficulty remembering it and so be tempted to write it down.
- In general, computer-generated password schemes have a history of poor acceptance by users.

Password Selection Strategies

Reactive password checking:

- This strategy is one in which the system periodically runs its own password cracker to find guessable passwords.
- The system cancels any passwords that are guessed and notifies the user.
- This tactic has a number of drawbacks: It is resource intensive if the job is done right.

Password Selection Strategies

Proactive password checking:

- This is the most promising approach to improve password security.
- A user is allowed to select his or her own password. However, at the time of selection, the system checks to see if the password is allowable and, if not, rejects it.
- Such checkers are based on the philosophy that, with sufficient guidance from the system, users can select memorable passwords from a fairly large password space that are not likely to be guessed in a dictionary attack.
- For example, the following rules could be enforced:
 - ▶ All passwords must be at least eight characters long.
 - ▶ In the first eight characters, the passwords must include at least one each of uppercase, lowercase, numeric digits, and punctuation marks.

Zipf's Law in Passwords

- D. Wang, H. Cheng, P. Wang, X. Huang, and G. Jian, “Zipf’s Law in Passwords,” **IEEE Transactions on Information Forensics and Security**, vol. 12, no. 11, pp. 2776–2791, Nov 2017.

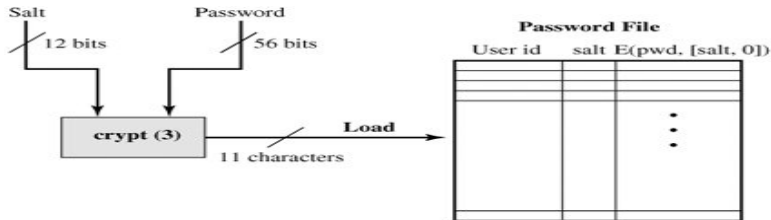
Biometrics and Fuzzy Extractor

- Let $\mathcal{M} = \{0, 1\}^v$ denote a finite v -dimensional metric space of biometric data points, $d : \mathcal{M} \times \mathcal{M} \rightarrow \mathbb{Z}^+$ a distance function, which can be used to calculate the distance between two points based on the metric chosen, l the number of bits of the output string, and t the error tolerance, where $\mathbb{Z}^+$ represents the set of all positive integers.
- The fuzzy extractor is a tuple $(\mathcal{M}, l, t)$, which is composed of the following two algorithms, called *Gen* and *Rep*:
 - ▶ **Gen:** It is a probabilistic algorithm, which takes a biometric information $B_i \in \mathcal{M}$ as input, and then outputs a secret key data $\sigma_i \in \{0, 1\}^l$ and a public reproduction parameter τ_i , where $Gen(B_i) = \{\sigma_i, \tau_i\}$.
 - ▶ **Rep:** This is a deterministic algorithm, which takes a noisy biometric information $B'_i \in \mathcal{M}$ and a public parameter τ_i and t related to B_i , and then it reproduces (recovers) the biometric key data σ_i . In other words, we have $Rep(B'_i, \tau_i) = \sigma_i$ provided that the condition $d(B_i, B'_i) \leq t$ is met.

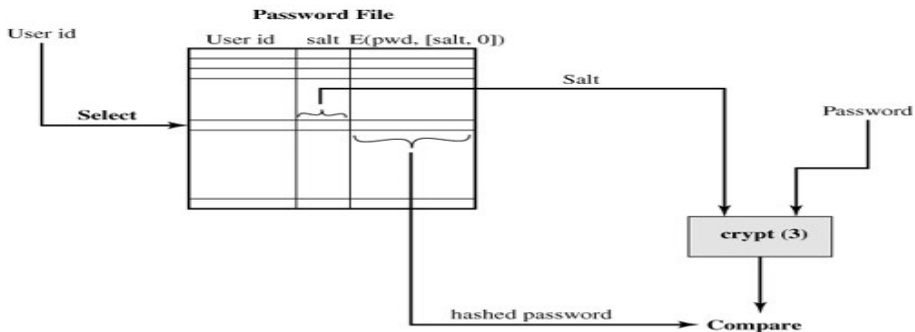
Biometrics and Fuzzy Extractor

- Vanga Odelu, **Ashok Kumar Das**, and Adrijit Goswami. "A Secure Biometrics-Based Multi-Server Authentication Protocol using Smart Cards," in ***IEEE Transactions on Information Forensics and Security***, Vol. 10, No. 9, pp. 1953 - 1966, 2015. (2015 SCI Impact Factor: 2.441) [This article is one of the top 50 most frequently downloaded documents for Popular Articles (June 2015 - June 2016)]

Password Management in UNIX



(a) Loading a new password



(b) Verifying a password